

Universal Property Management & Multi-Structure Evolution (SMP 2.0)

Comprehensive Technical Implementation Plan & Architectural Evolvment Guide

1. Goal & Problem Statement

Current Limitations

Currently, the codebase (`Laravel 11` + `CoreUI Blade` + `Vite`) is hardcoded around **Flats / Apartments**:

- **Database Schema:** Tables (`blocks` , `flats` , `flat_types` , `flat_documents`) and columns (`flat_no` , `floor_no` , `total_flats`) assume multi-story apartment towers.
- **Maintenance & Billing:** Maintenance rates are strictly fixed per `FlatType` (`owner_maintenance_fee` vs `rental_maintenance_fee`), ignoring square footage area (`sq. ft.`), commercial surcharges, or varying plot sizes.
- **UI/UX & Terminology:** Hardcoded labels ("Flat Management", "Block Management", "Floor No.", "Flat Documents") confuse or alienate users managing commercial complexes, bungalows, tenements, or row houses.
- **Occupant Profiles:** `residents` assumes individuals (`owner` / `rental`), lacking fields for Commercial Businesses, Corporate Tenants, GSTINs, or Trade Licenses.

Evolved System Capabilities (What We Will Enable)

1. **Commercial Complexes & IT Parks:** Supports Shops, Offices, Showrooms, Kiosks, and Food Courts with area-based (`per sq. ft.`) maintenance billing, GST compliance, and commercial signboards.
2. **Tenements, Villas & Bungalows:** Supports Plot-based structures without floor numbers, tracking plot area vs. built-up area and garden/clubhouse zones.
3. **Row Houses:** Supports lane/street groupings, party-wall notes, and common street infrastructure billing.
4. **Mixed-Use Societies:** Allows a single society to contain Residential Towers (Wing A & B), Commercial Shops (Wing C / Ground Floor Shopping Arcade), and Row Houses (Lane 1-4) simultaneously.

2. User Review Required & Critical Decisions

IMPORTANT: Database Schema Strategy (Backward Compatible vs. Renaming)

To preserve existing data and minimize disruptive migrations, we propose **Option A (Non-Breaking Polymorphic Expansion & Aliasing)**:

- Keep the `blocks` and `flats` table names (or alias `flats` to `units` in models), but **add polymorphic structure columns** (`unit_type`, `area_sqft`, `plot_no`, `calculation_method`) and nullable constraints where appropriate (e.g., `floor_no` becomes nullable for row houses/villas).
- **Alternative Option B**: Rename tables (`flats` ? `property_units`, `flat_types` ? `unit_categories`). Option A is recommended for speed and backward compatibility, while Option B is cleaner long-term if doing a major version release.

TIP: Dynamic Terminology Engine

We will introduce a **Dynamic Terminology Setting** in `Setting::defaults()`. The UI will automatically adjust its vocabulary based on `society_property_type`:

- If `commercial_complex`: "Wing" ? **"Tower/Zone"**, "Flat" ? **"Shop/Office"**, "Resident" ? **"Occupant/Business"**.
- If `rowhouse_villa`: "Wing" ? **"Sector/Phase/Lane"**, "Flat" ? **"Villa/Row House"**, "Floor No" ? **"Plot No"**.

3. Open Questions for Alignment

1. **Schema Renaming vs. Model Aliasing**: Do you prefer keeping the table name `flats` while adding `unit_type`, `area_sqft`, etc. (which avoids touching every single `$table->foreignId('flat_id')` migration in existing modules), OR would you like us to create a formal migration renaming `flats` ? `units` across the entire database? (*Recommended: Model Aliasing & Column Expansion*).
2. **GST & Commercial Invoicing**: For commercial complexes, should we include GST calculation (`CGST + SGST / IGST`) directly on the `maintenance_bills` and PDF invoice slips when the unit is marked as `commercial`? (*Recommended: Yes, via toggle in Settings: `enable_commercial_gst`*).

4. Proposed Architectural Changes & Implementation Details

A. Database Schema & Migration Enhancements

[NEW] `enhance_blocks_table_for_multi_structure.php`

Adds structure classification to `blocks` (Wings/Towers/Sectors/Lanes):

- `block_type`: string enum/type (`residential_tower` , `commercial_tower` , `row_house_lane` , `villa_sector` , `shopping_arcade` , `mixed_use`).
- `label_type`: string (`Wing` , `Tower` , `Block` , `Sector` , `Lane` , `Phase`).
- Make `total_floor` nullable or default `0` (since tenements/villas don't have multi-floors).

[NEW] `enhance_flat_types_table_for_flexible_billing.php`

Expands `flat_types` (Unit Categories) with dynamic billing models:

- `category_type`: enum('residential', 'commercial', 'institutional', 'industrial').
- `calculation_method`: enum('fixed', 'per_sqft', 'per_syard', 'hybrid') (Default: `fixed`).
- `rate_per_sqft`: decimal(10, 2)->nullable()->default(0) (Maintenance rate per square foot for commercial/sized units).
- `commercial_surcharge_percentage`: decimal(5, 2)->default(0) (Surcharge applied on base rates for shops/offices in mixed societies).

[NEW] `enhance_flats_table_for_multi_unit_types.php`

Expands `flats` (Property Units) to support all physical structures:

- `unit_type`: string (`flat` , `shop` , `office` , `showroom` , `row_house` , `villa` , `tenement` , `plot`).
- `unit_name_or_number`: string (Virtual alias or additional column if `flat_no` needs alphanumeric support like `Shop-101` or `Plot-42B`).
- `floor_no`: Modify to `nullable()` (Row houses/villas have no floor number within a building).
- `area_sqft`: decimal(10, 2)->nullable() (Super built-up / carpet area for maintenance calculation).
- `plot_area_syards`: decimal(10, 2)->nullable() (Plot dimensions for tenements/villas).
- `electricity_meter_no`: string->nullable() , `water_meter_no`: string->nullable() .
- `has_commercial_license`: boolean->default(false) .

[NEW] `enhance_residents_table_for_commercial_occupants.php`

Expands `residents` to support Commercial Businesses & Corporate Tenants:

- `occupant_category` : `string` (`individual` , `company` , `partnership` , `retail_brand`).
- `company_name` : `string->nullable()` (Business/Shop Name e.g. "Reliance Retail Ltd.").
- `gst_in` : `string->nullable()` (GST Number of commercial tenant/owner for invoicing).
- `trade_license_no` : `string->nullable()` (Shop & Establishment License Number).
- `emergency_or_manager_contact` : `string->nullable()` .

B. Core Model & Domain Logic Adapters

[MODIFY] `app/Models/Setting.php`

Add multi-structure default settings and terminology helper methods:

```
public static function defaults(): array
{
    return array_merge(parent_defaults(), [
        'society_property_type' => 'flat_residential', // Options: flat_residential, commercial
        'ui_label_block' => 'Block / Wing',
        'ui_label_unit' => 'Flat',
        'ui_label_unit_plural' => 'Flats',
        'ui_label_resident' => 'Resident',
        'enable_area_based_billing' => '0',
        'enable_commercial_gst' => '0',
        'commercial_gst_percentage' => '18',
    ]);
}

// Dynamic terminology helper for Blade views
public static function label($key, $default)
{
    return self::get("ui_label_{$key}", $default);
}
```

[MODIFY] `app/Models/PropertyUnit.php` (or `Flat.php` alias)

Add calculated attributes and scopes for flexible maintenance calculation:

```

public function calculateMaintenanceFee($residentType = 'owner')
{
    $type = $this->flatType;
    if (!$type) return 0;

    $baseRate = ($residentType === 'rental') ? $type->rental_maintenance_fee : $type->owner_maintenance_fee;

    if ($type->calculation_method === 'per_sqft' && $this->area_sqft > 0) {
        return round($this->area_sqft * $type->rate_per_sqft, 2);
    } elseif ($type->calculation_method === 'hybrid' && $this->area_sqft > 0) {
        return round($baseRate + ($this->area_sqft * $type->rate_per_sqft), 2);
    }

    // Apply commercial surcharge if unit or category is commercial
    if ($this->unit_type === 'shop' || $this->unit_type === 'office' || $type->category_type === 'commercial') {
        $surcharge = ($baseRate * $type->commercial_surcharge_percentage) / 100;
        return round($baseRate + $surcharge, 2);
    }

    return round($baseRate, 2);
}

```

C. Dynamic UI/UX Structure & Navigation Upgrades

- **sidebar.blade.php** : Make navigation items and icons adapt dynamically to the Society Type (e.g., displaying "Unit Management" / "Shop & Office Management" instead of hardcoded "Flat Management").
- **settings/index.blade.php** : Add a dedicated "Property & Structure Configuration" tab in General Settings to select structure type and customize UI vocabulary.
- **flats/index.blade.php & Forms**: Adapt form inputs based on structure capabilities (conditionally showing Area Sq. Ft., Plot Area, and hiding Floor No for villas/plots).
- **residents/create.blade.php** : Add Commercial Business Profile section (Company Name, GSTIN, Trade License No).

D. Billing Engine & Maintenance Generation Upgrade

- **MaintenanceBillController.php** : Upgrade monthly/quarterly maintenance bill generation to use `calculateMaintenanceFee()` and calculate/store `gst_amount` when commercial invoicing is enabled.

5. Verification & Testing Plan

Test Category	Verification Scope & Command	Expected Outcome
Automated Unit Tests	<code>php artisan test --filter=PropertyUnitCalculationTest</code>	Verifies accurate calculation of fixed rates, <code>per_sqft</code> rates, and commercial surcharge percentages.
Automated Feature Tests	<code>php artisan test --filter=DynamicPropertyStructureTest</code>	Verifies unit creation without floor numbers and commercial profile persistence with GSTIN.
Manual UI Flow	Settings ? Property & Structure Configuration	Verify dynamic terminology updates sidebar and form labels across all modules instantly.
Manual Billing Flow	Generate Monthly Maintenance Batch	Confirm itemized billing with area calculation and optional GST breakdown for commercial shops.